

VCE-IDC & LasVSim Technical Guide

This document provides a technical overview of the project, including the code structure, the controller interface you will implement, how evaluation metrics are calculated, and the key simulator APIs available to you.

1. Code Structure

Your work will be focused on a few key files. Understanding their roles is essential.

- `VCE-IDC/example_run/run_qx.py`:
 - This is the main executable script for running your agent.
 - It contains `QxTester`, which holds the simulation loop.
 - The `get_action` method in this class is the "bridge" between the simulator and your agent. It's responsible for calling the simulator API to get data and passing it to your agent's `update` method.
 - `VCE-IDC/gops/rule_model/planner.py`:
 - **This is the most important file for you.**
 - It contains the `Agent` class.
 - Your primary task is to implement the logic inside the `Agent.update` method.
 - `VCE-LasVSim/lasvsim_env/env/env_base.py`:
 - This is the backend environment file. You **should not** modify this.
 - Its `step` function (shown below) is what defines the simulation-agent interaction, calls the simulator, and **calculates the metrics** for safety and compliance based on the simulation results.
 - This file defines the `StepMetrics` and `EpisodeMetrics` data classes.
-

2. Controller Interface: `Agent.update`

You are required to implement your planning and control logic within the `Agent.update` method found in `VCE-IDC/gops/rule_model/planner.py`.

The method must compute and return the optimal action for the current timestep. The action should be specified as:

- **Longitudinal acceleration** (in m/s^2)
- **Steering angle** (in rad)

Method Signature and Data

Here is the interface you will be working with:

```
class Agent(object):
    def update(self, all_vehicle_info, all_veh_moving_info, all_veh_base_info,
               step, position_info, moving_info, reference_path, action, movement_signal):
```

```

# 1. Update your agent's internal state (self.update_state(...))
# YOUR LOGIC HERE

# 2. Make a decision on the target path (self.decision(...))
# YOUR LOGIC HERE

# 3. Generate a trajectory with your planner (self.planner(...))
# YOUR LOGIC HERE

# 4. Compute control commands with your controller
(self.Controller(...))
# YOUR LOGIC HERE

# Example return (must return two floats)
acc_command = 0.0
steer_command = 0.0

return acc_command, steer_command

```

3. Vehicle Model

This project involves two primary vehicle models: a kinematic model used in the MPC controller and a dynamic model used for internal state prediction within the simulator environment.

Kinematic Bicycle Model

This model is utilized within `mpc_controller.py` and defines the vehicle's discrete-time kinematics. It employs 4th-order Runge-Kutta (RK4) integration.

Source: function `fdyn` in `VCE-IDC/gops/rule_model/mpc_controller.py`

Dynamic Bicycle Model

This model is defined in `math_utils.py` and is used for state prediction within the environment. It is a single-track dynamic model that considers mass, rotational inertia, and tire cornering stiffness.

Source: function `ego_predict_model` in `VCE-LasVSIM/lasvsim_env/utils/math_utils.py`

4. Evaluation Metrics Calculation

You do not calculate the metrics yourself. The environment does this automatically in `env_base.py`. It is important to understand *how* they are calculated.

Per-Step Calculation

At every simulation step, the `env_base.py:step` function does the following:

1. Calls the simulator: `step_info = self.simulator.idc_step(...)`.
2. Judges the result: `terminated, truncated, done_info = self.judge_done(step_info["step_res"])`.
3. The `done_info` dictionary contains boolean flags like `event_collision`, `event_regionout`, and `event_traffic_light_violation`.
4. These flags are immediately used to set the metrics for the *current step*:

```

# In env_base.py:
step_metrics = self.episode_metrics.start_new_step()

# ... simulator step ...

# Safety metric update
step_metrics.set_safety_metrics(
    collision=done_info["event_collision"],
    out_of_bounds=done_info["event_regionout"] or done_info["event_mapout"],
)
# Compliance metric update
step_metrics.set_compliance_metrics(
    traffic_light_violation=done_info["event_traffic_light_violation"],
)

# Comfort metric update
action_diff = np.abs(self.lasvsim_context.ego.action -
self.lasvsim_context.ego.last_action)
action_abs = np.abs(self.lasvsim_context.ego.action)
step_metrics.set_comfort_metrics(action_diff, action_abs)

# ...

self.episode_metrics.finalize_current_step()

```

Episode-Level Aggregation

The `EpisodeMetrics` class (defined in `metrics.py`) stores a list of all `StepMetrics` and calculates the final scores:

- `get_safety_score()`: Returns `0.0` if **any** `step.collision` or `step.out_of_bounds` was `True` during the episode. Otherwise, returns `1.0`.
- `get_compliance_score()`: Returns `0.0` if **any** `step.traffic_light_violation` was `True`. Otherwise, returns `1.0`.
- `get_racing_score()`: Returns the `total_steps * dt` (where `dt = 0.1`).
- `get_comfort_score()`: Returns a dictionary containing the `mean` of all `action_diff` and `action_abs` values recorded during the episode.

5. Key Simulator APIs

You can access the simulator API via `env.simulator` inside the `get_action` function (in `run_qx.py`). You can pass this `simulator` object to your `Agent` if needed.

The following is a **curated list of the most essential APIs** you will likely need for this project.

Vehicle & Obstacle APIs

- `get_vehicle_id_list()`: Gets all vehicle IDs in the scene.
- `get_vehicle_position(id_list)`: Gets position, heading (`phi`), and lane info for a list of vehicles.
- `get_vehicle_moving_info(id_list)`: Gets kinematic info (longitudinal speed `u`, lateral speed `v`, yaw rate `w`) for a list of vehicles.

- `get_vehicle_base_info(id_list)` : Gets base info (like `length` and `width`) for a list of vehicles.
- `get_vehicle_collision_status(vehicle_id)` : Checks if a specific vehicle is in a collision state.

Navigation & Planning APIs

- `get_vehicle_reference_lines(vehicle_id)` : **(Crucial)** Gets the available reference paths/lanes for the ego vehicle.
- `get_idc_vehicle_nav(vehicle_id)` : (Used in `run_qx.py`) Gets navigation info, including the `next_movement_id`.
- `get_movement_signal(movement_id)` : (Used in `run_qx.py`) Gets the current traffic light signal (`current_signal`) for a given movement.

Road Map APIs

- `get_lane_center_line(lane_id)` : Gets the waypoints for a lane's center line.
- `get_lane_speed_limit(lane_id)` : Gets the speed limit for a specific lane.
- `get_lane_successor(lane_id)` : Gets the next lane(s) connected to this one.

Visualization APIs (Debugging)

We highly recommend using these to visualize your agent's decisions in the LasVSim replay.

- `set_vehicle_local_paths(vehicle_id, paths_data)` : Visualizes multiple planned paths (e.g., your planner's candidates).
- `set_vehicle_planning_info(vehicle_id, planning_path)` : Visualizes a single, chosen path (e.g., the trajectory sent to your controller).
- `set_vehicle_extra_metrics(vehicle_id, metrics_data)` : Displays custom text (e.g., "State: Following") in the simulation replay.